

PharmaCast

Pharmaceutical Demand Forecasting
Decision Support System

Comprehensive Project Report

Prepared for: Soorya
March 2026

BZ / CY / CZ Pharmaceutical SKU Segments

1. Project Overview

PharmaCast is a **Decision Support System (DSS)** for pharmaceutical demand forecasting. It targets the most difficult-to-forecast SKU segments (BZ, CY, CZ in the ABC-XYZ classification) and uses a combination of **baseline statistical methods** and **machine learning models** enhanced with **external signal data** to improve forecast accuracy and optimize safety stock levels.

The system generates a **self-contained HTML dashboard** (no server required) that provides interactive visualizations, risk analysis, model recommendations per SKU, safety stock optimization, and a built-in chatbot assistant.

Key Numbers at a Glance

Aspect	Value
Total SKUs	40 (8 per signal pool)
Signal Pools	5 (AQI, Monsoon, Temperature, Wedding, Google Trends)
Time Period	36 months (May 2022 - April 2025)
Data Records	1,440 (40 SKUs x 36 months)
ML Features	12
Candidate ML Models	4 (Gradient Boosting, Random Forest, Extra Trees, Ridge)
Baseline Models	2 (3-month SMA, Exponential Smoothing)

2. Problem Statement

In pharmaceutical supply chains, certain SKU categories are notoriously hard to forecast:

- **B-class / C-class SKUs** (medium-to-low revenue items) receive less attention than high-value A-class SKUs but still carry significant service-level requirements.
- **Y / Z variability classes** have erratic or intermittent demand patterns that simple forecasting methods handle poorly.

The BZ, CY, and CZ segments sit at the intersection of these challenges. PharmaCast addresses this by:

1. Incorporating **external signals** (air quality, weather, cultural events, search trends) that correlate with demand shifts in specific drug categories.
2. Using **per-pool ML model selection** so each signal group gets the best-fit algorithm.
3. Providing **actionable insights** through a dashboard with risk flags, model recommendations, and safety stock optimization.

3. Project Architecture

File Structure

```
DemandForecasting2/
  main.py          <- Orchestrator: runs the 3-step pipeline
  data_generator.py <- Synthetic data generation (40 SKUs, 36 months, 5 signals)
  forecasting.py    <- Baseline + ML forecasting pipeline
  dashboard.py      <- Self-contained HTML dashboard generator (~3000 lines)
  dashboard.html    <- Generated output (open in browser)
  requirements.txt  <- Python dependencies
```

Execution Flow

```
main.py
|
|-- [Step 1] data_generator.generate_all_data(seed=42)
|   |-- Generate 36 monthly dates (May 2022 - Apr 2025)
|   |-- Generate external signals (AQI, Temp, Precipitation, Wedding, Google Trends)
|   |-- Generate 40 SKU sales with signal-correlated demand
|   |-- Output: sales_df (1440 rows), signals_df (36 rows), metadata_df (40 rows)
|
|-- [Step 2] forecasting.run_forecasting_pipeline(sales_df, signals_df)
|   |-- Compute baseline forecasts (3-month SMA + Exponential Smoothing)
|   |-- Engineer 12 ML features
|   |-- Train 4 ML models per signal pool
|   |-- Select best model/ensemble per pool via validation
|   |-- Calculate per-SKU metrics (MAPE, FA, error std)
|   |-- Compute safety stock (baseline vs ML)
|   |-- Generate 3-month future forecasts with confidence intervals
|   |-- Output: results_df, metrics_df, future_df, model_log
|
|-- [Step 3] dashboard.generate_html_dashboard(metrics_df, results_df, future_df, signals_df)
|   |-- Prepare all data as embedded JSON
|   |-- Detect signal anomalies
|   |-- Build HTML with Plotly.js charts + interactive tabs + chatbot
|   |-- Output: dashboard.html (self-contained, ~300KB)
```

4. Data Generation Module

File: data_generator.py

4.1 Signal Pools

The system models 5 real-world signal categories, each linked to 8 pharmaceutical SKUs:

Pool	SKU Prefix	Drug Categories	Signal Driver	Correlation Factor
AQI	AQI-001 to AQI-008	Respiratory, antihistamine, bronchodilator	Air Quality Index	$0.15 \times (\text{aqi} - 120) / 100$
Monsoon	MON-001 to MON-008	Anti-malarials, ORS, anti-diarrhoeals	Precipitation (mm)	$0.20 \times (\text{precip} - 50) / 200$
Temperature	TMP-001 to TMP-008	Electrolytes, dermatological, heat-stroke	Temperature (C)	$0.18 \times (\text{temp} - 28) / 10$
Wedding	WED-001 to WED-008	Vitamins, nutraceuticals, supplements	Wedding season flag	$0.25 \times \text{wedding_flag}$ (binary)
GoogleTrends	GTR-001 to GTR-008	Symptom-linked (paracetamol, cough, etc.)	Google Trends index	$0.12 \times (\text{trends} - 40) / 30$

4.2 Complete SKU List (40 SKUs)

AQI Pool:

Salbutamol Inhaler, Cetirizine 10mg, Montelukast 10mg, Budesonide Inhaler, Levocetirizine 5mg, Fluticasone Spray, Theophylline 300mg, Chlorpheniramine 4mg

Monsoon Pool:

ORS Sachets, Loperamide 2mg, Chloroquine 500mg, Artemether-Lumefantrine, Zinc Dispersible 20mg, Metronidazole 400mg, Doxycycline 100mg, Racecadotril 100mg

Temperature Pool:

Electrolyte Powder, Calamine Lotion, Sunscreen SPF50, Glucose-D Powder, Hydrocortisone Cream, Prickly Heat Powder, Isotonic Drink Mix, Aloe Vera Gel

Wedding Pool:

Multivitamin Tablets, Biotin 10000mcg, Omega-3 Capsules, Vitamin C 1000mg, Collagen Powder, Iron + Folic Acid, Calcium + D3, Protein Supplement

GoogleTrends Pool:

Paracetamol 500mg, Azithromycin 500mg, Ivermectin 12mg, Dolo 650mg, Vitamin D3 60K, Zinc 50mg, Cough Syrup 100ml, Throat Lozenges

4.3 External Signal Generation

Signal	Range	Seasonality
AQI	30 - 500	Peaks in winter (Oct-Feb), poor air quality
Temperature	15 - 45 C	Indian climate: peak May-Jun, low Dec-Jan
Precipitation	0 - 300 mm	Monsoon peaks Jul-Sep
Wedding Flag	0 or 1	Active in Nov-Dec and Apr-May
Google Trends	0 - 100	Baseline + seasonal + 15% random spike probability

4.4 ABC-XYZ Classification

Target segments: BZ, CY, CZ only (the hardest to forecast)

Segment	ABC (Value)	XYZ (Variability)	Base Demand Range
BZ	B (Medium value)	Z (Highly erratic)	300 - 800 units/month
CY	C (Low value)	Y (Moderate variation)	80 - 350 units/month
CZ	C (Low value)	Z (Highly erratic)	80 - 350 units/month

Distribution across 40 SKUs: 15 BZ, 15 CY, 10 CZ (cycled per pool as ["BZ", "CY", "CZ"][i % 3]).

4.5 Demand Patterns

Three synthetic demand patterns are used:

1. **Intermittent:** 85% normal periods, 15% near-zero periods (20-50% of base) -- simulates stockout-prone items
2. **Erratic:** Normally distributed around base with high coefficient of variation ($\sigma = 0.3 \times \text{base}$)
3. **Seasonal:** Sinusoidal pattern with a random peak month per SKU

Final demand for each SKU per month:

```
demand = max(0, round(pattern_demand + signal_correlation_effect))
```

4.6 Output DataFrames

sales_df (1,440 rows):

date, sku_id, sku_name, signal_pool, abc_class, xyz_class, abc_xyz, demand

signals_df (36 rows):

date, aqi, temperature, precipitation, wedding_flag, google_trends

metadata_df (40 rows):

sku_id, sku_name, signal_pool, abc_class, xyz_class, abc_xyz

5. Forecasting Pipeline

File: forecasting.py

5.1 Constants

Constant	Value	Purpose
TRAIN_MONTHS	30	Months 1-30 used for training
TEST_MONTHS	6	Months 31-36 held out for final evaluation
VAL_MONTHS	6	Months 25-30 used for model selection
Z_SCORE	1.65	95% service level for safety stock
LEAD_TIME_DAYS	30	Replenishment lead time

5.2 Layer 1: Baseline Statistical Forecasts

3-Month Simple Moving Average (SMA):

$$\text{forecast}[t] = \text{mean}(\text{demand}[t-3], \text{demand}[t-2], \text{demand}[t-1])$$

Exponential Smoothing (alpha = 0.3):

$$\text{level}[t] = 0.3 \times \text{demand}[t] + 0.7 \times \text{level}[t-1]$$

$$\text{forecast}[t] = \text{level}[t-1]$$

Both serve as features for ML and as comparison benchmarks.

5.3 Layer 2: ML Feature Engineering

12 features are engineered for each data point:

#	Feature	Source
1	`baseline_forecast`	3-month SMA output
2	`exp_smoothing_forecast`	Exponential smoothing output
3	`aqi`	External signal
4	`temperature`	External signal
5	`precipitation`	External signal
6	`wedding_flag`	External signal (binary)
7	`google_trends`	External signal
8	`month_sin`	$\sin(2 \pi \times \text{month} / 12)$ -- cyclical encoding
9	`month_cos`	$\cos(2 \pi \times \text{month} / 12)$ -- cyclical encoding
10	`demand_lag1`	Demand at t-1
11	`demand_lag2`	Demand at t-2
12	`demand_rolling_std`	3-month rolling standard deviation of demand

5.4 Layer 3: Multi-Model ML Pipeline

4 Candidate Models:

Model	Algorithm	Key Hyperparameters
Gradient Boosting	GradientBoostingRegressor	150 trees, depth 4, lr 0.1, subsample 0.8
Random Forest	RandomForestRegressor	200 trees, depth 6, min_samples_leaf 3
Extra Trees	ExtraTreesRegressor	200 trees, depth 6, min_samples_leaf 3
Ridge Regression	Ridge	alpha = 1.0

5.5 Model Selection Process (Per Signal Pool)

This is a critical design choice: **model selection happens per signal pool**, not globally. Each pool has different demand dynamics driven by different signals, so the best model may differ across pools.

Step-by-step:

- Split data per pool:**
 - Train set: months 1-24 (24 months)
 - Validation set: months 25-30 (6 months)
 - Test set: months 31-36 (6 months, held out)
- Train all 4 models** on the train set (months 1-24)
- Evaluate all 4 models** on the validation set (months 25-30) using MAPE
- Rank models** by validation MAPE (lower is better)
- Retrain all models** on the full training data (months 1-30, train + validation combined)
- Build weighted ensemble** of top 3 models:
 - Weights = inverse MAPE: $\text{weight}[i] = (1 / \text{MAPE}[i]) / \sum(1 / \text{MAPE}[j])$
 - Lower MAPE = higher weight in ensemble
- Compare ensemble vs best single model** on validation set:
 - If ensemble MAPE < best single MAPE: use ensemble
 - Otherwise: use best single model
- Generate final predictions** on test set (months 31-36) using chosen model/ensemble

5.6 Per-SKU Metrics (Test Period)

For each of the 40 SKUs, the following metrics are computed over the test period (months 31-36):

Metric	Formula
MAPE	$\text{mean}(\frac{ \text{actual} - \text{forecast} }{\text{actual}})$
Forecast Accuracy (FA)	$\max(0, 100 - \text{MAPE})$
Improvement	$\text{ML_FA} - \text{Baseline_FA}$
Error Std	$\text{std}(\text{actual} - \text{forecast})$
MAPE Volatility	$\text{std}(\text{individual APE values})$
Risk Score	$(100 - \text{ML_FA}) + \text{MAPE_Volatility}$

5.7 Safety Stock Calculation

$$\text{Safety Stock} = Z \times \text{sigma_forecast_error} \times \sqrt{\text{Lead_Time_days}}$$

Where:

$Z = 1.65$ (95% service level)

$\text{sigma} = \text{std}(\text{actual} - \text{forecast})$ over test period

Lead Time = 30 days

Therefore: $SS = 1.65 \times \sigma \times \sqrt{30} = 1.65 \times \sigma \times 5.48 \sim 9.04 \times \sigma$

ML models with better accuracy produce lower forecast error standard deviation, which directly reduces required safety stock. The **reduction percentage** measures how much working capital is freed:

$\text{Reduction \%} = (\text{Baseline_SS} - \text{ML_SS}) / \text{Baseline_SS} \times 100$

5.8 Future Forecasting (3-Month Ahead)

For each SKU, the system generates point forecasts for the next 3 months (months 37-39) with 95% confidence intervals:

Point Forecast = mean(last 6 months demand) + small noise

Lower CI = Point - 1.96 x forecast_std

Upper CI = Point + 1.96 x forecast_std

6. Dashboard Module

File: dashboard.py (~3000 lines)

Output: dashboard.html (self-contained, ~300KB)

The dashboard is a **single HTML file** with embedded Plotly.js charts, CSS styling, and JavaScript logic. No server or external dependencies needed -- just open in a browser.

6.1 Design System

- **Background:** Warm off-white (#f5f5f0)
- **Cards:** Borderless white with soft shadows (no visible borders)
- **Colors:** Muted palette -- Green (#2d8a4e), Red (#bf4342), Amber (#b86e00), Blue (#2c6fce)
- **Typography:** Apple system fonts, 96px hero metric, clear hierarchy
- **Style:** Sentence case throughout, no emojis, human-readable language
- **Charts:** Plotly toolbar hidden (displayModeBar: false)
- **Border Radius:** 20px on all cards

6.2 Tab Structure

The dashboard has **4 main tabs** plus a floating chat assistant:

Tab 1: Overview

Purpose: "How well are my forecasts performing?"

Components:

- **Hero Metric** (96px): Overall improvement percentage (e.g., "+8.9%")
- **KPI Cards** (8 cards): Total SKUs, Avg Baseline FA, Avg ML FA, Avg Improvement, SKUs Improved, At-Risk SKUs, Highest Risk Score, Signal Anomalies
- **Intelligence Briefing:** Summary of key findings with amber left-border accent and action pill buttons
- **Pool Filter Pills:** Click to filter all charts by signal pool
- **Scatter Plot:** Baseline FA vs ML FA per SKU (above diagonal = ML wins)
- **Bar Chart:** Accuracy by signal pool (Baseline vs ML grouped bars)
- **Donut Chart:** Model selection distribution (how many SKUs use each model)
- **Accuracy Distribution:** Histogram/density showing FA spread

Tab 2: Risk Flags

Purpose: "Which SKUs need attention?"

Components:

- **Headline:** "N SKUs need attention" (count of FA < 70%)
- **Top 3 Risk Insight Cards:** Colored left-border stripes (red/amber), showing the worst performers with context about why they are at risk
- **Risk Data Table:** Sortable table with SKU ID, name, pool, class, FA%, improvement, risk score, volatility, demand
- **Risk Score Tooltip:** Info icon explaining the formula: Risk Score = (100 - FA) + MAPE Volatility
- **Anomaly Alerts:** Signal-based warnings (e.g., "AQI spike detected: 310")
- **Risk Scatter:** Visual plot of risk scores

Tab 3: Model Recommendations

Purpose: "Which model should I use for each SKU?"

Components:

- **SKU Selector:** Searchable dropdown with risk indicator dots (red/amber/green)
- **SKU Detail Panel:** Name, ID, pool, ABC-XYZ class, status badge
- **Model Comparison Bar Chart:** Test-period MAPE for all 4 candidate models per selected SKU
- **Time Series Chart:** Actual vs Baseline vs ML forecast over the test period
- **Residual Analysis:** Forecast errors plotted over time
- **Future Forecast:** Next 3 months with confidence interval bands
- **Methodology Section:** 7 expandable steps explaining the entire pipeline with model details

Tab 4: Safety Stock

Purpose: "How much safety stock do I need?"

Components:

- **Comparison Cards:** Total Baseline SS vs Total ML SS vs Total Reduction
- **What-If Simulator:** Interactive sliders for:
 - Service Level (90% / 95% / 99%)
 - Lead Time (15 / 30 / 45 / 60 days)
 - Error Adjustment factor
- **Top 10 Reduction Bar Chart:** SKUs with highest safety stock savings
- **Formula Display:** Explains the safety stock formula with all parameters

Chat Assistant (Floating Panel)

Purpose: Natural language query interface for the dashboard data

Access: Floating button in bottom-right corner, opens as a slide-over panel

Capabilities:

- **SKU Lookup:** Ask about any SKU by ID or name -- returns FA, improvement, model recommendation, safety stock, future forecast
- **Pool Analysis:** Ask about any signal pool -- returns pool-level metrics, best/worst SKU
- **Risk Queries:** "Which SKUs are at risk?" -- returns top 5 worst performers with risk scores
- **Performance Queries:** "Best performing pool?" -- returns rankings
- **Safety Stock Queries:** "Safety stock savings?" -- returns total and per-SKU reduction
- **Diagnosis:** "Diagnose root causes" -- cross-references risk SKUs with signal anomalies
- **Formula Explanations:** "What is MAPE?" / "How is risk score calculated?" -- returns definitions
- **Future Forecasts:** "Forecast for AQI-001?" -- returns next 3-month projections

Implementation: Pattern-matching NLP engine with keyword-based intent routing (no external API calls).

7. Technology Stack

Component	Technology
Language	Python 3.x
Data Processing	NumPy, Pandas
Machine Learning	Scikit-learn (GradientBoosting, RandomForest, ExtraTrees, Ridge)
Visualization	Plotly.js 2.27.0 (embedded in HTML)
Frontend	HTML5, CSS3, Vanilla JavaScript
Deployment	Self-contained HTML file (no server required)

Dependencies (requirements.txt):

```
numpy>=1.24.0  
pandas>=2.0.0  
scikit-learn>=1.3.0
```

8. How to Run

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run the pipeline
python main.py

# 3. Open the generated dashboard
# Open dashboard.html in any modern web browser
```

The pipeline runs in 3 steps:

1. **Data Generation:** Creates synthetic sales + signal data (deterministic with seed=42)
2. **Forecasting:** Runs baseline + ML pipeline, computes metrics and safety stock
3. **Dashboard Generation:** Builds self-contained HTML file

Output is `dashboard.html` -- a single file you can open directly in Chrome, Firefox, Safari, or Edge.

9. Key Metrics and Formulas

MAPE (Mean Absolute Percentage Error)

$$\text{MAPE} = (1/n) \times \sum(|\text{actual} - \text{forecast}| / \text{actual}) \times 100$$

Lower is better. Represents average percentage error.

Forecast Accuracy (FA)

$$\text{FA} = \max(0, 100 - \text{MAPE})$$

Higher is better. Represents how close forecasts are to actuals.

Risk Score

$$\text{Risk Score} = (100 - \text{ML_FA}) + \text{MAPE_Volatility}$$

Higher means more risky. Combines forecast inaccuracy with forecast inconsistency.

Safety Stock

$$\text{SS} = Z \times \text{sigma_error} \times \text{sqrt}(\text{Lead_Time_days})$$

$Z = 1.65$ for 95% service level

Lower forecast error (sigma) means less safety stock needed.

Ensemble Weighting

$$\text{weight}[i] = (1 / \text{MAPE}[i]) / \sum(1 / \text{MAPE}[j]) \text{ for } j \text{ in top } 3$$

Models with lower MAPE get proportionally higher weight.

At-Risk Threshold

A SKU is "at risk" when $\text{ML_FA} < 70\%$ ($\text{MAPE} > 30\%$)

10. Summary Table

Aspect	Detail
Project Name	PharmaCast - Demand Forecasting DSS
SKUs	40 pharmaceutical SKUs across 5 signal pools
Target Segments	BZ, CY, CZ (hardest to forecast)
Data Period	36 months synthetic (May 2022 - Apr 2025)
Baseline Methods	3-month SMA + Exponential Smoothing ($\alpha=0.3$)
ML Models	Gradient Boosting, Random Forest, Extra Trees, Ridge
Model Selection	Per-pool validation (months 25-30) + ensemble of top 3
Test Evaluation	Hold-out months 31-36
Safety Stock	$Z=1.65$ (95% SL), Lead Time=30 days
Output	Self-contained HTML dashboard with 4 tabs + chatbot
External Signals	AQI, Temperature, Precipitation, Wedding Season, Google Trends
Key Innovation	Signal-pool-specific ML model selection with weighted ensemble

This report covers the complete structure and methodology of the PharmaCast project. For code-level details, refer to the inline docstrings and comments in each Python module.